

# From Suspicious Signals to Crashes: Guiding Bug-Driven GUI Testing via Code-Inspired Tracing

**MENGZHUO CHEN\***, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**ZHE LIU\***, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**CHUNYANG CHEN**, Technical University of Munich, Germany

**JUNJIE WANG<sup>†</sup>**, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**YUEKAI HUANG**, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**BOYU WU**, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**JUN HU**, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

**QING WANG<sup>†</sup>**, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, China

Mobile applications frequently suffer from crash bugs that are triggered under specific GUI interaction sequences. Existing automated GUI testing techniques mainly emphasize increasing coverage through diverse exploration strategies, but they often fail to reach the precise interaction contexts that lead to crashes, resulting in low bug detection efficiency. This paper proposes TraceDroid, a novel automated GUI testing approach that leverages suspicious code-level signals to guide dynamic exploration. Instead of treating static analysis as an independent detection method, TraceDroid uses heuristic rules distilled from real crash reports to detect suspicious code segments, associate them with GUI widgets, and collect code-level interaction signals. It then constructs the Activity Transition Graph (ATG), performs rough path generation, and employs LLM-based executable path completion to produce a set of suspicious paths. Finally, TraceDroid executes these paths through global path planning, local path generation, and execution-aware monitoring to efficiently expose crashes. We evaluate TraceDroid on 70 real crash bugs across 42 open-source apps, comparing it with 15 state-of-the-art baselines. TraceDroid achieves the best performance, with a recall of 77%, exceeding the best baseline by 28%, while maintaining comparable or higher coverage. Furthermore, TraceDroid successfully

\*Both authors contributed equally to this research.

<sup>†</sup>Corresponding author.

Authors' Contact Information: **Mengzhuo Chen**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, mengzhuo.happy@gmail.com; **Zhe Liu**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, liuzhe181@mails.ucas.edu.cn; **Chunyang Chen**, Technical University of Munich, Munich, Germany, chun-yang.chen@tum.de; **Junjie Wang**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, junjie@iscas.ac.cn; **Yuekai Huang**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, boyu\_wu2021@163.com; **Boyu Wu**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, huangyuekai18@mails.ucas.ac.cn; **Jun Hu**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, hujun@iscas.ac.cn; **Qing Wang**, Institute of Software and State Key Laboratory of Intelligent Game, Chinese Academy of Sciences, China and University of Chinese Academy of Sciences, Beijing, China, wq@iscas.ac.cn.



This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/).

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE151

<https://doi.org/10.1145/3808158>

detects 21 previously unknown crash bugs in 116 popular Google Play apps, of which 15 have been fixed and 6 confirmed by developers, demonstrating its effectiveness in real-world scenarios.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Mobile App, Large language model, GUI testing

# ACM Reference Format:

Mengzhuo Chen, Zhe Liu, Chunyang Chen, Junjie Wang, Yuekai Huang, Boyu Wu, Jun Hu, and Qing Wang. 2026. From Suspicious Signals to Crashes: Guiding Bug-Driven GUI Testing via Code-Inspired Tracing. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE151 (July 2026), 24 pages. <https://doi.org/10.1145/3808158>

## 1 Introduction

Mobile applications have become an integral part of daily life, and ensuring their reliability has become a critical concern for both developers and researchers. Software crash bugs often have a huge impact on our software usage, which can abruptly terminate an app’s execution, potentially leading to data loss, broken workflows, or negative user experiences. Research shows that crash bugs often occur under specific interaction conditions, many of which require a precise sequence of GUI operations to activate, making crash bugs inherently difficult to uncover. To detect crash bugs, researchers and practitioners typically employ automated GUI testing techniques, which simulate user interactions and observe application behavior during execution to capture crash bugs.

In recent years, automated GUI testing has continuously evolved with more sophisticated exploration strategies. Model-based methods [25, 28, 47, 51, 53, 63, 71, 78, 80, 83] aiming to increase activity or code coverage under the assumption that more coverage correlates with more bugs found. Learning-based [42, 55] and LLM-driven methods [31, 40, 46, 49, 67, 81, 82] imitate human interaction patterns to generate more diverse and realistic action sequences. These methods expand coverage and interaction diversity, but they are not explicitly designed to prioritize crash-prone behaviors. As a result, crash bugs that require specific or long interaction sequences are often undetected. As shown in Fig.1, in the open-source app *LibreTube*[7], a crash is triggered after following a relatively long path of at least ten steps, involving multiple functionalities such as downloading a video, switching between playback modes, navigating back, and finally clicking the audio conversion button. The crash does not occur if the user selects the “Yes” option in the intermediate dialog. Instead, it arises when the user dismisses the dialog via the back button and then proceeds to the audio function. Although each step is valid on its own, the large number of possible operations makes it infeasible to cover all combinations. This also requires GUI testing not only to achieve coverage, but also to determine which paths should be prioritized for exploration.

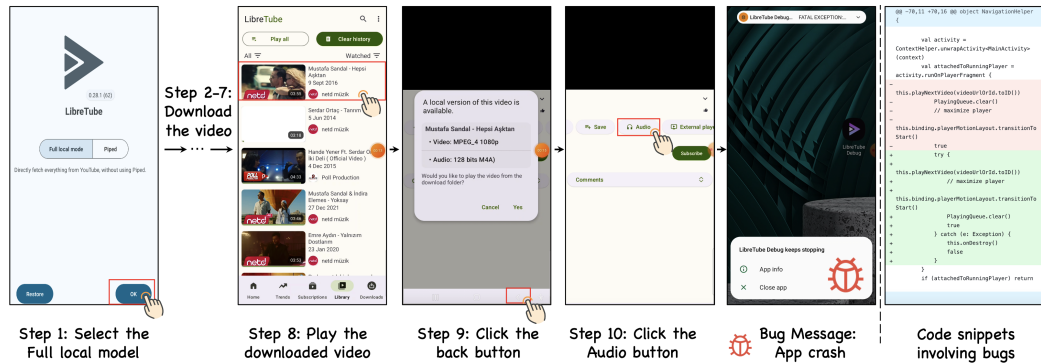


Fig. 1. Example of a crash bug and its bug code

To address this difficulty, some researchers have begun exploring bug-driven testing paradigms. BugHunter [20] retrieves historical crash knowledge from similar apps to guide GUI testing, but it depends heavily on the availability of high-quality bug data and performs best when relevant apps exist. Other researchers construct knowledge graphs from historical bug reports of the same app [65, 66, 72], encoding past failures into structured knowledge to generate new test cases. However, they require significant manual effort and are limited to a single app, restricting their generalizability. In this work, our goal is to design a proactive testing approach that leverages code-level patterns as guidance signals, helping exploration prioritize paths that are more likely to expose crashes. Although static analysis [14, 24, 35–38, 50, 56, 57, 59, 61, 74, 79] has been studied for detecting potential bugs, it faces fundamental challenges in rule design. If the rules are strict, many bug patterns are missed because they do not always deterministically lead to bugs. If the rules are relaxed, the tools generate an overwhelming number of warnings, making the results impractical for manual inspection. Therefore, rather than treating code analysis as a bug detector, we use it as a provider of exploration guidance signals. By mapping these suspicious code signals to GUI widgets and execution paths, our approach allows dynamic exploration to prioritize suspicious contexts first, while still maintaining balanced coverage.

We propose TraceDroid, an automated GUI testing approach that uses code signals to guide dynamic exploration and prioritize bug detection. TraceDroid consists of three modules. TraceDroid first collects suspicious code-level signals. We design a set of heuristic rules derived from our empirical study of real-world crashes, which capture representative suspicious patterns and associate them with corresponding GUI widgets. By further backtracking event handlers and call chains, this module extracts early code-level signals that serve as seeds for subsequent analysis. Second, TraceDroid links the code-level signals to generate execution paths. TraceDroid constructs a global Activity Transition Graph (ATG) and performs rough path generation by linking suspicious widgets to navigation flows. To handle missing or obfuscated attributes, we employ LLM-based reasoning for executable path completion, ensuring that each suspicious interaction is aligned with a feasible cross-activity route. Third, TraceDroid adopts suspicious paths to guide action generation. It combines global path planning, which prioritizes exploration and reduces redundant traversals, with local path generation to adapt to runtime UI variations. An execution-aware monitor continuously checks whether the suspicious paths are exercised as intended and dynamically adjusts the exploration strategy.

We evaluate the effectiveness of TraceDroid on 70 crash bugs collected from 42 open-source apps. Compared with 15 widely used and state-of-the-art automated GUI testing baselines, TraceDroid achieves the highest bug detection performance, with a recall of 77%, outperforming the best baseline by 28%. The activity and code coverage of TraceDroid are also comparable to or higher than those of the baselines. To further understand the role of each sub-module, we conduct ablation experiments that confirm the necessity and effectiveness of our design. We also evaluate the usefulness of TraceDroid by testing unseen apps from Google Play. Among 116 popular apps, TraceDroid successfully detects 21 new crash bugs, of which 15 have already been fixed and 6 have been confirmed by developers.

The contributions of this paper are as follows:

- We propose TraceDroid<sup>1</sup>, a novel automated GUI testing approach that proactively explores suspicious paths by combining code-level signals with dynamic exploration. It collects suspicious code-level signals, generates suspicious paths through ATG construction and path completion, and guides dynamic execution to efficiently detect bugs. (Section 3)

<sup>1</sup>We release the source code, dataset, and experimental results on our website <https://github.com/TheMystery123/TraceDroid>.

- We conduct an empirical study on crash bugs and analyzed their distribution. We also design heuristic rules for different types for future research.
- Effectiveness and usefulness evaluation of the TraceDroid in widely used open-source apps on Google Play with practical bugs detected and confirmed (Section 5).

## 2 Motivational Study

### 2.1 Data Collection

The experimental dataset is collected from GitHub, which has a great number of crash issue reports. We randomly select 300 apps with 5783 issue reports. Two filtering criteria are applied: (1) whether the report describes a crash bug, and (2) whether the report can be mapped to specific code modifications in the linked commit. Two authors manually check the selected apps and issue reports to ensure they have crash bugs. The third author rechecks the results until a consensus is reached. A total of 1639 crash bugs with bug reports are collected. The filtered dataset is then used for manual inspection and classification of crash root causes.

### 2.2 Categorizing Crash Bugs

During the manual label process, we observed that crash bugs in mobile applications exhibit recurring patterns across different projects. Categorizing these patterns can help us design heuristic rules and guide dynamic exploration more effectively. According to the card sorting method [62], we classify the crash bugs into five major categories, each containing several subcategories that reflect common triggering contexts and functional scenarios. The detailed information is as follows.

**Null Pointer Exceptions (29%):** As shown in Fig. 2 (a), this category refers to crashes caused by accessing null objects without proper validation. It includes *GUI binding failures*, where views are accessed before data loading is complete; *asynchronous data access*, where callbacks reference uninitialized or released objects; and *collection/array null access*, where data structures are accessed without null checks. These issues frequently occur in functionalities involving data loading and UI updates, such as news feeds, media galleries, or chat lists.

**Runtime Exceptions and Error Handling (27%):** As shown in Fig. 2 (b), this category covers crashes triggered by missing or insufficient exception handling during critical operations. Subcategories include *UI thread violations*, where UI elements are updated from non-main threads; *resource misuse*, such as invalid resource IDs or missing assets; and *type/encoding errors*, including type casting failures or character encoding mismatches. These bugs are common in background operations like uploads, scans, or data backups.

**External Integration and Compatibility (19%):** As shown in Fig. 2 (c), this category includes crashes resulting from incompatibilities with external APIs, third-party libraries, or platform-specific behaviors. Subcategories include *third-party API changes* that are not adapted in time; *platform-specific behaviors* that differ across Android versions; and *permission/manifest configuration issues*, such as incorrect permission declarations or forced removal of system-level permissions. These crashes often occur in features that depend on external services or hardware integrations.

**Concurrency and Resource Management (14%):** As shown in Fig. 2 (d), this category refers to crashes caused by improper handling of concurrent processes or system resources. It includes *repeated component start*, where services or activities are started multiple times in quick succession; *state check omissions*, where component states are not verified before operations; and *non-atomic variable access*, leading to inconsistent states in concurrent scenarios. Such crashes often appear in functionalities involving shared resources, such as content sharing or user status updates.

**Database Operation Issues (11%):** As shown in Fig. 2 (e), this category involves crashes related to database misuse or exceeding system limitations. It includes *data size limit exceeded*, such as

storing large BLOBs beyond SQLite constraints; *incorrect query/comparator*, where violations of comparator contracts cause crashes during sorting; and *ORM misuse*, where APIs from frameworks like Room are incorrectly applied. These issues are prevalent in data-heavy applications such as note-taking or accounting software.

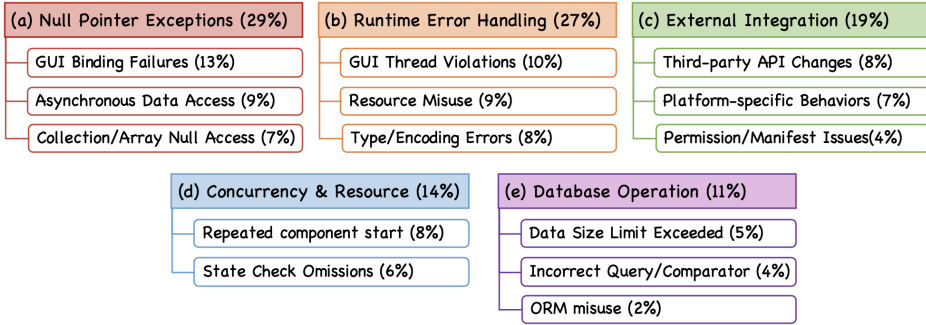


Fig. 2. Example of 5 categories of crash bug

### 3 Approach

We propose TraceDroid, an automated GUI testing approach designed to proactively explore abnormal execution contexts by combining code-level signals with dynamic exploration. The core idea is that although GUI testing is the practical way to detect crashes, unguided exploration often fails to achieve the precise interaction sequence that triggers crashes. TraceDroid utilizes lightweight code analysis not to directly detect crashes, but to identify suspicious code-level signals that may indicate unstable behavior and use them as guidance for GUI exploration.

As shown in Fig. 3, TraceDroid consists of three main modules: *Suspicious Code-level Signals Collection*, *Context-aware Suspicious Path Generation*, and *Dynamic Execution Guided by Suspicious Paths*. The first module applies heuristic rules to detect suspicious code patterns, associates them with corresponding GUI widgets, and collects their code-level interaction signals. The second module constructs the Activity Transition Graph (ATG) of the target app, generates rough paths, and employs LLM-based reasoning to complete executable paths, producing a set of suspicious paths. The third module conducts dynamic execution guided by these suspicious paths, integrating global path planning with local path generation, while an execution-aware monitoring mechanism validates execution and records crashes. Meanwhile, the suspicious paths discovered by TraceDroid can also be integrated into existing LLM-based GUI testing approaches as prompt-level hints, potentially enhancing their bug detection ability.

#### 3.1 Suspicious Code-Level Signals Collection

The purpose of this module is to generate inspiration for GUI testing by tracing from suspicious code fragments to their corresponding GUI widgets and collecting their *code-level interaction signals*. Based on the empirical study in Section 2, we design heuristic rules that capture five representative categories of crash-inducing patterns. Applying these rules to the app's source code and configuration files produces a set of suspicious code segments. Each segment is then associated with candidate GUI widgets by analyzing layout files and event-binding relations. Finally, we extract the interaction signals from method calls and callback chains that are linked to these widgets. The output of this module is a set of *suspicious code-level signals*, each consisting of a suspicious code fragment, the associated GUI widget, and its triggering signals. These signals serve as seeds for the path generation module in Section 3.2, where they are further aligned with execution-level paths.

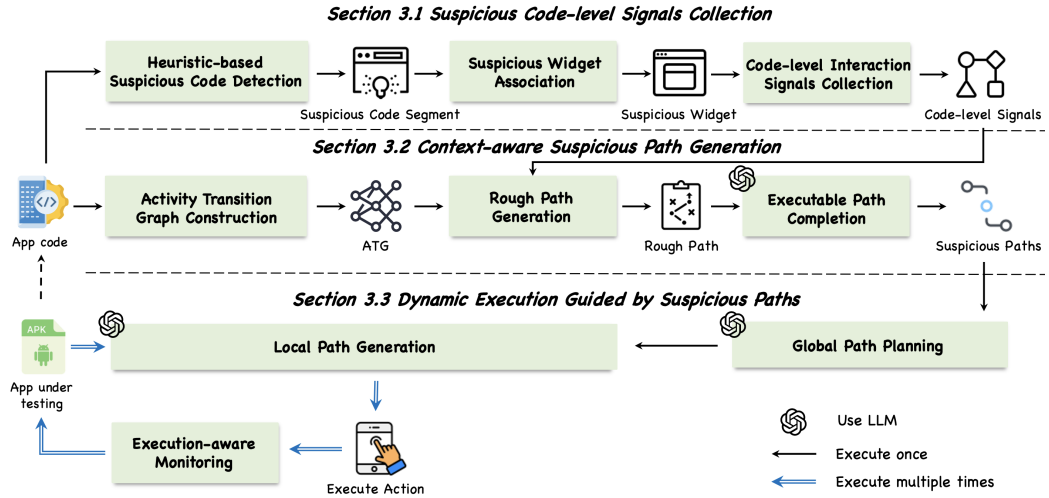


Fig. 3. Overview of TraceDroid

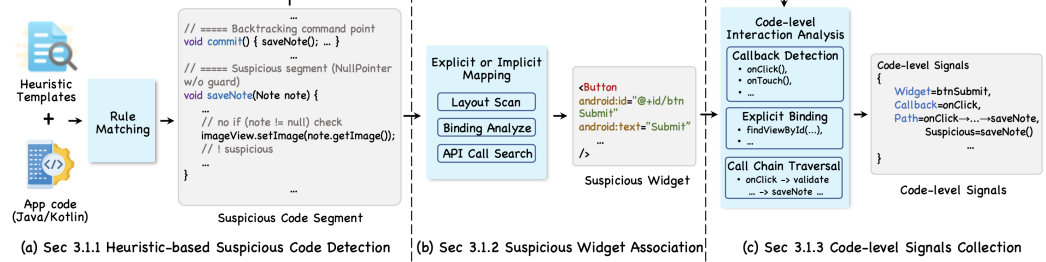


Fig. 4. Example of suspicious code-level signals collection

**3.1.1 Heuristic-Based Suspicious Code Detection.** The first step of our approach is to automatically locate code segments as shown in Fig. 4 (a), that exhibit suspicious patterns. Based on the findings of our motivational study in Section 2, we designed a set of heuristic detection rules that capture five representative categories of crash-inducing patterns: *NullPointerException*, *RuntimeError*, *ExternalIntegration*, *Concurrency*, and *DBOperation*. In total, we implemented 73 fine-grained rules, with 10-18 sub-rules per category, together with several variants to account for implementation differences across projects. The detailed list of rules is made publicly available in our GitHub repository<sup>1</sup> for reproducibility.

The rules are designed by following two principles. First, each rule encodes empirical patterns distilled from real-world bug reports, ensuring that the rule set is grounded in practical evidence. Second, rules consider *contextual conditions* rather than isolated statements. For instance, dereference operations (e.g., `obj.method()` or `obj.field`) are only flagged as suspicious if they appear along a control-flow path where no null-check (e.g., `if (obj != null)`) dominates the operation. Similarly, a call to `setText()` is only flagged when it is reachable from a background thread without a dispatcher to the UI thread, indicating a potential runtime violation. This context-sensitive design distinguishes our rules from simple syntactic pattern matching and reduces false positives. Across all rules, a single rule involves between one and three contextual conditions (average of 1.9 conditions per rule). For *NullPointerException* crashes (16 rules), where contextual conditions mainly capture missing null-dominance checks or premature dereference of objects before proper initialization. For *RuntimeError* crashes (18 rules), their contextual conditions frequently encode whether

UI-modifying operations are reachable from background threads, or whether lifecycle callbacks are invoked without required state guards. For *ExternalIntegration* crashes (14 rules), whose contextual conditions focus on platform and environment dependencies, such as missing permission checks, unavailable external services, or incompatible API usage. For *Concurrency* crashes (13 rules), whose contextual conditions emphasize temporal and state-related properties, including repeated lifecycle invocations, missing synchronization, or reuse of stale component states. For *DBOperation* crashes (12 rules), where contextual conditions mainly describe database-specific execution constraints, such as unclosed cursors, incomplete transactions, or invalid access after database closure.

To implement the detection, we parse all Java/Kotlin source files into Abstract Syntax Trees (ASTs) and traverse the control-flow and data-flow graphs. Each rule is applied to the AST representation of methods and callback handlers, scanning not only the target operation but also the surrounding conditions and preceding statements. Layout files and resource identifiers (e.g., `R.id.*`) are also parsed to enrich the analysis, since many suspicious behaviors are tied to specific widgets or event handlers. For concurrency-related rules, the analysis further considers the temporal proximity of repeated lifecycle calls (e.g., consecutive `startService()` invocations) and checks whether the corresponding termination or state validation is missing. The output of this step is a set of suspicious code segments annotated with their rule category and location.

**3.1.2 Suspicious Widget Association.** After suspicious code segments are detected, the next step is to associate them with the corresponding GUI widgets. We design a multi-dimensional matching strategy that integrates information from three sources: layout XML files, event binding relations, and API call semantics. This strategy ensures that suspicious segments are mapped to the correct widgets even when the connections are indirect, implicit, or spread across multiple files. As shown in Fig. 4 (b), our implementation covers both explicit mappings (e.g., direct references to `R.id.*`) and implicit mappings (e.g., dynamically inflated layouts or inherited listeners).

First, we parse layout XML files that are connected to the suspicious segment. We scan for layout inflation calls, including `setContentView(R.layout.*)` and `inflate(R.layout.*, ...)`. For each relevant layout, we extract all widgets and record their attributes: (1) the component ID from the `android:id` attribute, (2) the widget type from the XML tag name (e.g., `Button`), (3) textual information from the `android:text` field or linked string resources, and (4) the relative position in the XML hierarchy, which preserves parent-child relationships.

Second, we analyze event binding information. Suspicious segments often occur inside event listeners, so we track calls that register event handlers, including `setOnClickListener()`, `setOnLongClickListener()`, `setOnTouchListener()`, etc. We further resolve indirect bindings where the listener registration is delegated through helper methods or abstract classes. These bindings are linked to widget IDs by analyzing the parameters of registration calls and their surrounding control-flow scope.

Third, we incorporate API call semantics. For example, if a suspicious segment invokes `setText()` on a `TextView`, the mapped widget should be of type `TextView` or its subclass. This semantic verification ensures type-level consistency between suspicious operations and candidate widgets.

**3.1.3 Code-Level Interaction Signals Collection.** Once suspicious code segments are associated with candidate widgets, the next step is to collect code-level interaction signals that indicate how these widgets may trigger the suspicious operations. As shown in Fig. 4 (c), we extract key interaction-related signals from the code that can later serve as seeds for suspicious path generation.

We begin by parsing the method that contains the suspicious code into its Abstract Syntax Tree (AST) and control-flow graph (CFG). The backward traversal searches for enclosing or upstream methods that are recognized as event callbacks, which include framework-defined callbacks (`onClick()`, `onItemSelected()`, `onTouch()`, `onLongClick()`), adapter callbacks (`getView()`), and

system lifecycle methods (`onActivityResult()`, `onResume()`). When a suspicious statement is located inside such a callback, we infer that the corresponding widget interaction can directly trigger it.

If the connection is indirect, we extend the analysis to cover two additional scenarios. First, we resolve *explicit bindings*, where event listeners are dynamically registered in the code. We analyze calls such as `findViewById()` followed by `setOnClickListener()` (or variants like `setOnTouchListener()`, `addTextChangedListener()`). The integer parameter of `findViewById()` is traced back to a resource ID defined in the layout XML, which is then matched to a widget entry. Second, we address *implicit bindings*, where handler methods are invoked through indirection, such as XML-declared `android:onClick` attributes or reflection-based invocations. For these cases, we scan layout files for attributes that reference method names and match them to methods in the AST. Reflection calls are resolved conservatively by analyzing string literals passed to `Class.getMethod()` or `Method.invoke()`, recording all feasible method targets. The backtracking also considers multi-step call chains. For example, a suspicious segment in `saveNote()` may be invoked by `onClick()`, but only after passing through helper methods (e.g., `validate()` → `commit()` → `saveNote()`). We traverse the call graph until reaching a GUI entry point (a callback registered to a widget or lifecycle). This traversal is bounded to avoid infinite loops, and recursive calls are resolved by summarizing the call site once.

The final output of this step is a set of structured *code-level interaction signals*, consisting of: (1) the widget ID, type, and layout file in which it is defined, (2) the sequence of event-handling methods leading to the suspicious segment, and (3) the relative position of the suspicious code within this chain. These signals are used as seeds for the suspicious path generation module in Section 3.2, where they are aligned with executable paths of the app.

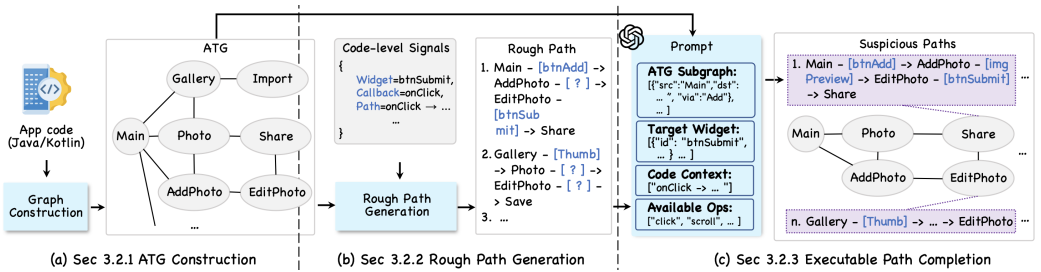


Fig. 5. Example of context-aware suspicious path generation

### 3.2 Context-Aware Suspicious Path Generation

After obtaining code-level interaction signals from the first module, the second module aims to map these signals to concrete GUI widgets and align them with executable navigation paths. The key challenge here is that static analysis often provides only partial information, and the same widget may appear in multiple activities or layouts. To address this, TraceDroid first performs *Activity Transition Graph (ATG) Construction* by analyzing both manifest declarations and navigation code, ensuring that all possible inter-activity transitions are captured. It then conducts *Rough Path Generation*, which leverages identifiers, textual properties, type compatibility, and binding relations to associate code-level signals with candidate widgets and approximate their navigation paths within the ATG. The output of this module is a set of *suspicious paths* that connect code-level signals with their corresponding GUI widgets and cross-activity navigation traces, serving as direct guidance for the dynamic execution module in Section 3.3.

**3.2.1 Activity Transition Graph Construction.** The first step of this module is to construct a global Activity Transition Graph (ATG) that captures all feasible navigation flows within the app. We begin by parsing the *AndroidManifest.xml* file to enumerate all declared activities and services. For each activity, we record its name, intent filters (including actions, categories, and data types), and launch modes. This step establishes the node set of the ATG as shown in Fig. 5 (a).

Next, we analyze the app's source code to extract inter-activity transitions. Explicit transitions are identified by locating invocations of APIs such as *startActivity()*, *startActivityForResult()*, and *startService()*. When the *Intent* instantiation explicitly specifies a target class (e.g., *new Intent(this, TargetActivity.class)*), we directly add a directed edge from the caller activity to *TargetActivity*. We also account for indirect intent construction, such as when the target component is passed as a variable or obtained from a helper method, by propagating symbolic references along the control-flow graph until the target class is resolved.

Implicit transitions are extracted by cross-referencing manifest-declared intent filters with code-level intent usage. For example, when an activity declares an intent filter with action *android.intent.action.VIEW* and category *android.intent.category.DEFAULT*, any *sendBroadcast()* or *startActivity()* call with a matching action-category pair is linked to that activity. This matching process allows us to capture navigation edges that are not explicitly defined in the code but are nevertheless valid at runtime.

In addition to activity-to-activity transitions, we also incorporate fragment-based navigation. We detect fragment transactions (e.g., *FragmentManager.add()*, *replace()*, *show()*) and model fragments as intermediate nodes within the ATG. Edges are added from the host activity to its fragments, and between fragments, when a transaction replaces one with another. This extension allows the ATG to capture intra-activity navigation that is critical in modern Android applications.

Unlike traditional ATG construction that only records activity-level edges, our approach further annotates each transition with the widgets or triggers discovered in static analysis. For instance, if an invocation of *startActivity()* is guarded by a button click, the corresponding edge is annotated with the widget's ID and type (e.g., *btnAdd*, *ImagePreview*). In cases where partial information is missing, such as when no *id* or *text* can be resolved, the annotation is left blank. Similarly, when static analysis reveals an actionable widget but fails to link it to a specific transition edge, we conservatively attach the widget to the activity node itself.

**3.2.2 Rough Path Generation.** After obtaining the enriched ATG with annotated but often incomplete widget information, the next step is to generate *rough paths* that connect code-level signals to candidate navigation traces. Some edges are annotated with precise identifiers (e.g., *R.id.btnAdd*), while others may only reveal a callback name or widget type, and certain components remain entirely unlinked. To bridge these gaps, we design a matching strategy that systematically integrates both code-level and UI-level evidence as shown in Fig. 5 (b).

Specifically, for each code-level signal *widget*, *callback*, *path* obtained in Module 3.1.3, we attempt to match it with candidate widgets discovered in the ATG. Some are extracted from static code (IDs, listener registrations), others from resource files or runtime hierarchy dumps (text labels, widget types, structural positions). To tolerate incompleteness, we use a scoring across four dimensions: (1) *ID consistency*. When an explicit ID is available, we normalize it (removing *R.id.* prefixes and aliases) and directly compare. (2) *Textual similarity*. When a label or content description is present, we compute string similarity between code constants and resource texts. For instance, a suspicious call *setText("Submit")* can be aligned with a widget in XML defined as *android:text="Submit"*. (3) *Type compatibility*. The widget type implied by the API usage (e.g., *setImage()*  $\Rightarrow$  *ImageView*) is cross-checked with the XML or hierarchy type. (4) *Callback linkage*. If the suspicious code resides

in a listener (e.g., `onClick()`), we check whether the candidate widget is registered with the same listener type.

A candidate widget is accepted if the combined score passes the threshold of 0.8. When multiple candidates qualify, we keep all of them for later disambiguation. Through this process, the ATG edges that were partially annotated become enriched with concrete or candidate widgets, forming *rough paths* that connect suspicious code-level signals to navigation traces. These rough paths still contain missing slots (e.g., “[?]” as shown in Fig. 5 (b)), where LLM-based reasoning will complete and refine them into executable suspicious paths.

**3.2.3 Executable Path Completion.** After obtaining rough paths from the previous stage, some edges are annotated with missing IDs, some widgets lack textual labels, and some navigations are unresolved. To turn these rough traces into executable navigation sequences, we employ an LLM to perform *executable path completion*. The model leverages both structural information from the ATG and contextual hints from the code-level signals to fill in the missing steps and produce realistic interaction paths.

For each rough path, we serialize a structured prompt that contains the following fields as shown in Fig. 5 (c). (1) ATG Subgraph. A two-hop neighborhood around the source and target activities, represented as a list of candidate transitions (e.g., [“src”: “Main”, “dst”: “AddPhoto”, “via”: “Button:id=btnAdd”, ... ]). (2) Target Widget. The suspicious widget to be triggered, with any available attributes such as id, text, or type. (3) Code-level Signals. The callback chain and local guards obtained from backtracking (e.g., `onClick` → `saveNote()`, with required conditions such as `note != null`). (4) Available Operations. The primitive actions executable on device (`click`, `scroll`, `input`, etc.) and selectors supported (id, text, xpath).

The LLM then outputs a candidate action sequence in JSON format, where each step specifies the activity, selector, operation, and optional params. To ensure reliability, we validate the generated paths using two lightweight checks: (1) ATG consistency: all inter-activity hops must exist in the subgraph or correspond to valid implicit intents. (2) Widget existence: each selector must correspond to an actual widget in the layout or view hierarchy dump. Only validated plans are retained as *suspicious paths*. Paths that share long prefixes are merged, allowing reuse across multiple suspicious contexts.

### 3.3 Dynamic Execution Guided by Suspicious Paths

This module uses the suspicious paths as explicit guidance while still retaining the flexibility to adapt to runtime conditions. The design consists of three complementary steps. First, a *Global Path Planning* phase schedules which suspicious paths should be explored and in what order, balancing high-risk prioritization with coverage considerations. Second, a *Local Path Generation* phase translates each suspicious path into concrete executable actions based on the current GUI state, resolving ambiguities such as multiple candidate widgets or layout variations. Third, an *Execution-aware Monitoring* phase observes runtime feedback, checking whether the intended suspicious path was successfully executed, whether it triggered abnormal behaviors, and whether recovery actions (e.g., restarting the app) are required.

**3.3.1 Global Path Planning.** Before execution starts, TraceDroid performs a one-time global scheduling that determines the order in which suspicious paths should be explored. The Activity Transition Graph (ATG) provides the structural backbone, but does not prescribe an execution sequence. This step therefore converts the unordered suspicious paths into an executable itinerary.

To achieve this, we design a structured prompt for the LLM that contains four elements: (1) an identifier explanation that clarifies the meaning of activities and widgets; (2) the extracted ATG edges, which encode all feasible activity transitions; (3) the list of candidate suspicious paths

```

# App Under Test: PhotoManager
# Current Goal: Explore suspicious paths derived from our analysis
# Identifier Explanation: Each item without brackets (e.g., Main, Gallery,
                        EditPhoto) denotes an Activity in the app. Items inside brackets (e.g.,
                        [btnAdd], [imgPreview], [btnSubmit]) denote ...

# ATG: [Main, Gallery], [Gallery, Photo], [Photo, EditPhoto] ...

# Candidate Suspicious Paths:
- Path 1: Main -> [btnAdd] -> AddPhoto -> [imgPreview] -> EditPhoto
- Path 2: Gallery -> [Thumb] -> Photo -> [btnEdit] -> EditPhoto -> ...
- Path 3: Import -> [FilePicker] -> EditPhoto -> [btnSubmit] -> ...

# Constraints:
- Start exploration from LauncherActivity
- Minimize backtracking cost
- Prefer batching within the same cluster
- Ensure all preconditions are satisfied

# Query: Based on the ATG and candidate suspicious paths:
1. Reorder the paths into an executable sequence.
2. Return the output strictly with fields: {"path": ..., "prep_prefix": ...}.

```

(a) Global Path Planning Prompt

```

# App Under Test: PhotoManager
# Global Suspicious Path: Main -> [btnAdd] -> AddPhoto -> [imgPreview] -> ...

# Long-term Memory (Global Progress):
- Planned Suspicious Paths:
  1. Main -> AddPhoto -> EditPhoto -> Share
  2. Gallery -> Photo -> EditPhoto -> ...
- Paths Completed: None so far

# Short-term Memory (Recent Steps):
- Step 1: Launched app, entered Main activity
- Step 2: Clicked [btnMenu] -> navigated to Main. ...

# Current UI (View Hierarchy Snapshot): Activity: AddPhotoActivity
- WidgetID: "1", Text/Label: "Camera", Type: Button, Action: Click
- ...

# Execution Constraints:
- Must follow the suspicious path defined in the Global Suspicious Path
- Each step must operate on one widget and ensure the widget exists

# Query: Based on the suspicious path, short-term memory, and the current UI
snapshot. Identify the next widget ("widget": "WidgetID:3", "action": "click")

```

(b) local Path Planning Prompt

Fig. 6. Example of prompt in global and local path planning

derived from widget mapping and backtracking; and (4) explicit constraints such as starting from the launcher activity, minimizing backtracking cost, batching paths within the same cluster, and ensuring all preconditions are satisfied.

The query at the end of the prompt explicitly instructs the LLM to reorder the candidate paths into an executable sequence and return the output in a strict JSON format. This structured format avoids ambiguity and allows TraceDroid to directly consume the LLM's plan. After receiving the output, TraceDroid validates each transition against the ATG using graph traversal. If the LLM proposes an unreachable hop, it is replaced by the deterministic shortest path. Missing or malformed fields are also repaired through lightweight deterministic checks. This validation ensures that the final itinerary is both syntactically correct and semantically executable, while the LLM provides high-level ordering that exploits locality and shared prefixes. The result of this step is a deterministic itinerary, where each suspicious path appears exactly once. This itinerary is then passed to the Local Path Generation phase, which translates each path into concrete interaction steps at runtime.

**3.3.2 Local Path Generation.** After global planning produces a high-level itinerary, TraceDroid must translate each scheduled suspicious path into concrete interaction steps that can be directly executed on the device. Since the planned paths are abstract and runtime UIs may contain multiple candidates or dynamic variations, local path generation provides step-by-step grounding with explicit memory and context management.

Each generation task is formulated as a structured prompt to the LLM. (1) Global Suspicious Path selected from global planning, which ensures the LLM operates within the intended navigation trajectory. (2) Long-term Memory (Global Progress) is the list of planned suspicious paths and their current execution status. This avoids revisiting already completed branches and ensures continuity across multiple paths. (3) Short-term Memory (Recent Steps) means the last few actions executed, which provides local execution context and prevents loops or redundant actions. (4) Current UI (View Hierarchy Snapshot) means the annotated GUI screenshot, including widget IDs, labels, types, and clickable/input attributes. These annotations serve as explicit grounding for possible actions.

The query then asks the LLM to generate the next single action in JSON format. By constraining output to one step per query and enforcing a fixed schema, TraceDroid prevents free-form responses and facilitates automatic replay on the device. Through this design, local path generation combines global trajectory alignment, explicit memory for long-term and short-term contexts, and accurate grounding in the current UI snapshot. Unlike conventional GUI testing that executes

pre-computed scripts or random interactions, this approach enables TraceDroid to adaptively resolve ambiguities at runtime while ensuring consistency with suspicious paths.

**3.3.3 Execution-Aware Monitoring.** The final step of dynamic execution is to monitor whether each suspicious path has been faithfully exercised and whether its execution exposes abnormal behaviors. Unlike conventional crash detection that relies solely on coarse-grained system logs, TraceDroid integrates runtime monitoring with both path context and UI state to provide fine-grained validation and robust recovery.

The monitoring process takes as input. (1) Scheduled Suspicious Path. The planned sequence of activities and widgets that should be executed. (2) Runtime Execution Trace. The sequence of actions actually performed, each paired with its corresponding view hierarchy snapshot. (3) System-level Feedback. Signals from logcat, tombstone files, exception stack traces, and process status.

During execution, each action is verified against the suspicious path. If a widget mismatch occurs (e.g., expected `btnSubmit` is absent due to a dynamic layout), the path is flagged as incomplete, and a fallback strategy continues from the nearest valid state. TraceDroid continuously observes abnormal behaviors across three categories: (1) Crash events. Uncaught exceptions or process terminations, identified by stack traces and exit codes. (2) Freeze or deadlock. Detected through repeated failed UI hierarchy dumps without progress. (3) Path deviations. Divergence between the intended suspicious path and the executed trace. After each path terminates, the monitor ensures the app is reset to a clean state. Crashed apps are relaunched automatically, while incomplete paths are retained for possible re-execution if resources allow.

### 3.4 Implementation

In our implementation, we utilize the GPT-4 Vision model<sup>2</sup> (gpt-4o) as MLLM to handle both textual and visual information during the testing process. It obtains the view hierarchy file of the current GUI page through UIAutomator [69] to extract text information of the input widgets. TraceDroid is implemented as a fully automated GUI testing tool, which uses or extends the following tools: VirtualBox and the Python pyvbox [4] for running and Android Debug Bridge (ADB) [2] for interacting with the app.

## 4 Experiment Design

- **RQ1: (Detection Effectiveness and Coverage)** How effective is TraceDroid in exposing crash behaviors and achieving testing coverage?

For RQ1, we evaluate the effectiveness of TraceDroid in detecting crash bugs and compare it with widely used and state-of-the-art automated GUI testing baselines.

- **RQ2: (Ablation Study)** What is the contribution of each (sub-)module of TraceDroid to the overall detection effectiveness and coverage?

For RQ2, we conduct ablation experiments by selectively disabling modules of TraceDroid to evaluate their impact on detection rate and coverage.

- **RQ3: (Usefulness Evaluation)** How useful is TraceDroid in real-world scenarios?

For RQ3, we use TraceDroid to automatically explore the app from Google Play, detect unseen bugs, and issue the detected bugs to the development team.

<sup>2</sup><https://platform.openai.com/docs/models/gpt-4o>

## 4.1 Dataset

**For RQ1 and RQ2**, the experimental dataset is derived from the same collection process described in our pilot study in Section 2, but using a disjoint set of apps. In detail, we randomly selected 150 different apps with 183 crash bug reports that do not overlap with those 300 in the pilot study, following the same app names to guarantee disjointness. We also keep the ones with at least one update after Jan. 2024 in 15 App categories.

Then, we manually reviewed all collected issue reports and filtered apps based on the following criteria: (1) All baseline tools can run on the app. (2) UIAutomator [69] can extract both the view hierarchy file and GUI screenshots. (3) The crash bugs can be reliably reproduced in the given app version. Finally, 42 apps with 70 crash bugs remain as the evaluation dataset for effectiveness and ablation studies. The distribution of these bugs across categories (*NullPointerException*, *RuntimeError*, *ExternalIntegration*, *Concurrency*, and *DBOperation*) follows the proportions observed in the pilot study, ensuring that the evaluation dataset reflects real-world crash characteristics.

To further characterize the complexity of the evaluated crashes, we manually inspected the reproduction steps of all 70 crash bugs. We find that the crash-triggering paths require between 4 and 16 GUI interactions, with an average of 9.1 steps. These interactions typically involve a mixture of navigation actions, form inputs, and repeated backtracking across multiple activities, rather than single-step or immediately failing operations. This confirms that the evaluated crashes are generally non-trivial and require multi-step interaction sequences to be triggered.

**For RQ3**, we further evaluate the usefulness of TraceDroid on real-world applications from Google Play. We randomly selected 200 popular apps across different categories, ensuring diversity in functionality and download volume. To ensure reproducibility and fair comparison, we only retain apps that satisfy the following constraints: (1) The apps can be successfully decompiled and their source code retrieved, so that our heuristic rules can be applied. (2) UIAutomator can obtain the view hierarchy and screenshots during testing. After filtering, we obtained a set of 116 apps as the evaluation target. For each detected crash bug, we created a detailed issue report including reproduction steps and submitted it to the corresponding developers through issue trackers or email. Developer confirmations and fixes are used as ground truth to assess usefulness.

## 4.2 Baselines

To demonstrate the advantage of TraceDroid, we compare it with 15 commonly used and state-of-the-art automated GUI testing techniques. We roughly divide them into random-/rule-based, model-based, and learning-based methods, to facilitate understanding. For random-/rule-based methods, we use Monkey [23] and Droidbot [41], Time-Machine [25]. For model-based methods, we use Ape [28], Fastbot [19], ComboDroid [71]. For learning-based methods, we use Humanoid [42] and Q-testing [55]. For LLM-based methods, we use AppAgent [81], GPTDroid [46], DroidAgent [82], AUITestAgent [31], VisionDroid [48], and Guardian [58]. For the bug-driven method, we use Bughunter [20], a bug-aware automated GUI testing approach that generates exploration paths guided by bug reports from similar apps, utilizing a combination of LLMs and RAG.

## 4.3 Experimental Setup

We implement TraceDroid based on the gpt-4o API which is released on the OpenAI website. We deploy the baselines and TraceDroid on a 64-bit Ubuntu 18.04 machine and evaluate them on Google Android 7.1 emulators. Following common practice [28, 41], we register separate accounts for each app that requires login and write the login scripts, and during testing reset the account data before each run to avoid possible interference. To ensure fair and reasonable use of resources, we set up the running time of each tool in one app to 60 minutes, which is widely used in other

Table 1. Per-category bug detection recall in RQ1.

| Metric   | NullPointerException | RuntimeError | ExternalIntegration | Concurrency | DBOperation | Overall |
|----------|----------------------|--------------|---------------------|-------------|-------------|---------|
| Recall   | 0.81                 | 0.83         | 0.71                | 0.70        | 0.71        | 0.77    |
| F1-score | 0.89                 | 0.91         | 0.83                | 0.82        | 0.83        | 0.87    |

GUI testing studies [26, 28, 41, 64]. We manually optimized the automated GUI testing tools based on the configuration suggestions provided by the method and applied them. We run each tool three times and obtain the average performance to mitigate potential bias.

#### 4.4 Evaluation Metrics

For bug detection, we use the number of bugs detected (#Bugs) by automated GUI testing tools and the recall rate (Recall) to assess the effectiveness of TraceDroid in identifying true bugs. Recall measures the ability of TraceDroid to identify all relevant bugs present within the app. To calculate recall, we compare the number of bugs detected by TraceDroid with the total number of bugs confirmed by developers in the issue reports. For testing coverage, we obtain the activity coverage and code coverage [13, 29, 42, 44, 71, 73], in which we treat the activities defined in the *AndroidManifest.xml* file of an Android app as the whole set of activities [3, 55, 63]. For bug detection time, we record the timestamp at which each crash bug is first triggered during the 60-minute testing window. If an app contains multiple crash bugs, each bug has an associated detection time. We compute the *average bug detection time* by taking the arithmetic mean of all detection times of the bugs that are successfully exposed by a tool within the app. For example, if three crash bugs are detected at 5, 18, and 42 minutes respectively, the average detection time is  $(5 + 18 + 42)/3 = 21.7$  minutes. If a bug is not detected within the allotted time, it is excluded from the average since its detection time is undefined.

## 5 Result

### 5.1 Performance of Bug Detection (RQ1)

We first analyze the bug detection performance of TraceDroid across different crash categories. Among the 54 detected bugs, NullPointerException (17), RuntimeError (15), ExternalIntegration (10), Concurrency (7), and DBOperation (5). As shown in Table 1, TraceDroid achieves relatively high recall and F1-score for all five categories, with recall values ranging from 0.70 to 0.83 and F1-scores ranging from 0.82 to 0.91. Among them, *RuntimeError* and *NullPointerException* crashes are detected with the highest effectiveness. In particular, *RuntimeError* achieves the highest F1-score (0.91), followed by *NullPointerException* (0.89), indicating that these crash types can be detected with both high recall and stable performance. This is mainly because crashes in these categories often exhibit relatively explicit code-level patterns and well-defined execution contexts. These patterns can be precisely localized by suspicious-code detection and effectively mapped to triggering GUI interactions, allowing TraceDroid to prioritize and reconstruct the corresponding crash-triggering paths.

We further present the bug detection performance of TraceDroid and baselines in Table 2. Across 42 apps with 70 confirmed crash bugs, TraceDroid achieves a recall of 77% (54/70), significantly outperforming all 15 baselines. It achieves 57% (0.77 vs. 0.49) recall higher compared with the best baseline (BugHunter) across these apps. In terms of the number of detected bugs, TraceDroid can still achieve a higher detection rate. We also compare the similarities and differences of the bugs between BugHunter and TraceDroid, and the results show that all bugs detected by BugHunter are also detected by TraceDroid. This indicates the effectiveness of TraceDroid in detecting crash bugs and helps to ensure app quality.

Table 2 also shows the average bug detection time of TraceDroid and the baselines. We can see that TraceDroid detects bugs significantly faster than all baselines. The average detection time

Table 2. Performance of TraceDroid and baselines

| Automated GUI testing | Bug detection |        | Average coverage |      | Average bug detection time |
|-----------------------|---------------|--------|------------------|------|----------------------------|
|                       | #Bugs         | Recall | Activity         | Code |                            |
| Random-based method   |               |        |                  |      |                            |
| Monkey                | 11            | 0.16   | 0.28             | 0.26 | 31.2                       |
| Droidbot              | 15            | 0.21   | 0.31             | 0.30 | 33.9                       |
| Time-Machine          | 19            | 0.27   | 0.34             | 0.33 | 29.7                       |
| Model-based method    |               |        |                  |      |                            |
| Fastbot               | 20            | 0.29   | 0.34             | 0.32 | 30.3                       |
| Ape                   | 18            | 0.26   | 0.33             | 0.31 | 32.5                       |
| ComboDroid            | 21            | 0.30   | 0.37             | 0.35 | 35.1                       |
| Learning-based method |               |        |                  |      |                            |
| Humanoid              | 20            | 0.29   | 0.35             | 0.33 | 33.1                       |
| Q-testing             | 18            | 0.26   | 0.32             | 0.31 | 29.6                       |
| LLM-based method      |               |        |                  |      |                            |
| AppAgent              | 21            | 0.30   | 0.38             | 0.35 | 28.9                       |
| GPTDroid              | 24            | 0.34   | 0.43             | 0.42 | 33.7                       |
| DroidAgent            | 20            | 0.29   | 0.38             | 0.36 | 35.2                       |
| AUITestAgent          | 23            | 0.33   | 0.41             | 0.38 | 32.4                       |
| VisionDroid           | 26            | 0.37   | 0.49             | 0.47 | 28.1                       |
| Guardian              | 27            | 0.39   | 0.51             | 0.49 | 27.7                       |
| Bug-driven method     |               |        |                  |      |                            |
| BugHunter             | 34            | 0.49   | 0.48             | 0.45 | 16.6                       |
| TraceDroid            | 54            | 0.77   | 0.52             | 0.50 | 9.3                        |

Notes: “#Bugs” is the number of the detected bugs from automated GUI testing tools.

of TraceDroid is 9.3 minutes, while the best baseline BugHunter requires 16.6 minutes, which means TraceDroid reduces 44% (9.3 vs. 16.6) of the time to trigger a crash. This again proves the efficiency of TraceDroid, which is valuable for saving more time for bug detection. Table 2 further reports the average activity coverage and code coverage of TraceDroid and the baselines. We can see that TraceDroid achieves 0.52 activity coverage and 0.50 code coverage across the 42 apps, which is comparable to BugHunter (0.48 and 0.45) and even slightly higher. Considering that TraceDroid is a GUI testing tool and the goal of TraceDroid is to detect more crash bugs, it can still achieve and exceed the optimal baseline in terms of testing coverage. This also indicates the effectiveness of TraceDroid in covering more activities and code regions. We further analyze the cost and latency overhead of TraceDroid to contextualize the reported detection time. The average detection time reported in Table 2 measures the elapsed time until a crash is first triggered during dynamic exploration, and does not include the one-time preprocessing overhead. In practice, the static analysis phase takes 0.9-1.8 minutes per app on average, dominated by AST and CFG construction, while the LLM-based reasoning stage introduces an additional latency of 40-70 seconds. As this preprocessing is performed only once before testing and does not scale with the exploration duration, its overhead is amortized over the entire testing process and does not affect the relative efficiency comparison across tools.

The advantages of TraceDroid are due to three aspects. First, TraceDroid identifies suspicious code segments and uses them to guide testing, which helps to focus exploration on error-prone areas. Second, TraceDroid excels at reconstructing complex triggering paths that involve repeated navigation and backtracking before reaching the crash, which are often missed by coverage-driven

Table 3. Contribution of different sub-modules (RQ2)

| Sub-module                                | #Bugs     | Recall      | Activity coverage | Code coverage | Detection time |
|-------------------------------------------|-----------|-------------|-------------------|---------------|----------------|
| <b>TraceDroid</b>                         | <b>54</b> | <b>0.77</b> | <b>0.52</b>       | <b>0.50</b>   | <b>9.3</b>     |
| <i>w/o Interaction Signals Collection</i> | 44        | 0.63        | 0.47              | 0.46          | 11.4           |
| <i>w/o Rough Path Generation</i>          | 47        | 0.67        | 0.47              | 0.45          | 12.2           |
| <i>w/o Executable Path Completion</i>     | 41        | 0.59        | 0.43              | 0.42          | 14.7           |
| <i>w/o Global Path Planning</i>           | 42        | 0.60        | 0.47              | 0.46          | 11.1           |
| <i>w/o Execution aware Monitoring</i>     | 43        | 0.61        | 0.45              | 0.44          | 11.8           |

baselines. Third, the suspicious path-guided execution ensures that these high-risk paths are explored with priority, significantly improving both effectiveness and efficiency in bug detection.

## 5.2 Ablation Study (RQ2)

We first study the effect of removing suspicious-code guidance entirely, by disabling the static analysis module and relying only on dynamic execution over the ATG. In this setting, the system explores the app using runtime execution and execution-aware monitoring, but without any suspicious code signals to guide path generation or prioritization. Under this configuration, the recall drops to 0.49, which is substantially lower than the full TraceDroid (0.77). This result indicates that dynamic execution alone, even when combined with execution-aware monitoring, is insufficient to reliably uncover crash-inducing behaviors without code-level guidance.

Table 3 further shows the performance of TraceDroid and its five variants, where each variant removes one key sub-module. For *w/o Interaction Signals Collection*, we remove the backtracking of suspicious code segments to their triggering callbacks and upstream operations, meaning suspicious code is only linked to widgets without considering interaction signals. For *w/o Rough Path Generation*, we disable the matching process and instead rely only on direct ID matching, which ignores cases where IDs are obfuscated or missing. For *w/o Executable Path Completion*, we remove the LLM-based completion, leaving only the raw ATG and static mappings without path refinement or disambiguation. For *w/o Global Path Planning*, we disable the LLM-based global scheduling and simply traverse the ATG in a breadth-first manner. For *w/o Execution-aware Monitoring*, we disable runtime monitoring of execution states and crashes, so the system only records completed paths without detecting incomplete or abnormal terminations.

From Table 3, we can see that TraceDroid’s bug detection recall, activity coverage, and code coverage are consistently higher than all other variants, demonstrating the necessity of each designed sub-module. Compared with TraceDroid, removing the LLM-based path completion (*w/o Executable Path Completion*) leads to the largest performance decline, i.e., 23% drop in recall (0.59 vs. 0.77) and 8-9% decline in coverage. This highlights the importance of context-aware reasoning to complete missing paths and consolidate noisy mappings. Removing rough path generation (*w/o Rough Path Generation*) also causes a noticeable decrease in recall, i.e., 13% drop (0.67 vs. 0.77), since many widgets cannot be accurately resolved by ID alone. Code-level interaction signals (*w/o Interaction Signals Collection*) also contribute significantly, with an 18% recall reduction (0.63 vs. 0.77), because without tracing upstream callbacks, suspicious code often cannot be tied to the correct triggering operations. Removing global path planning (*w/o Global Path Planning*) decreases recall by 22% (0.60 vs. 0.77), showing that the LLM-guided scheduling effectively reduces redundant exploration and prioritizes high-risk paths. Finally, removing execution-aware monitoring (*w/o Execution-aware Monitoring*) causes a 21% drop (0.61 vs. 0.77), since the tool cannot distinguish between successful and incomplete executions, leading to missed detections. These results confirm that each sub-module contributes to the overall effectiveness of TraceDroid. The LLM-based executable path completion is particularly crucial, as it bridges the gap between partial static

Table 4. Impact of different large language models on bug detection performance (RQ2)

| Different LLM       | #Bugs | Recall | Activity coverage | Code coverage | Detection time |
|---------------------|-------|--------|-------------------|---------------|----------------|
| TraceDroid (GPT-4o) | 54    | 0.77   | 0.52              | 0.50          | 9.3            |
| GPT-4o mini         | 48    | 0.69   | 0.47              | 0.46          | 12.4           |
| Gemini-1.5-Pro      | 51    | 0.73   | 0.50              | 0.48          | 10.5           |
| LLaMA-4             | 50    | 0.71   | 0.49              | 0.48          | 10.9           |
| Qwen2.5-72B         | 47    | 0.67   | 0.47              | 0.46          | 12.7           |

analysis results and runtime execution. Meanwhile, the other modules together ensure accurate signal extraction, path generation, efficient scheduling, and reliable detection of crash-inducing behaviors.

We further investigate the impact of different large language models on the performance of TraceDroid. As shown in Table 4, TraceDroid achieves the best performance when using GPT-4o, while still maintaining competitive recall when paired with other LLMs. Gemini-1.5-Pro and LLaMA-4 achieve recalls of 0.73 and 0.71, with comparable coverage, respectively, demonstrating that the proposed framework is compatible with different LLMs. We observe that weaker models are more likely to generate invalid or partially inconsistent interaction sequences. In contrast, GPT-4o shows stronger reasoning capability over mixed structural inputs and more stable adherence to the required output schema.

### 5.3 Usefulness Evaluation (RQ3)

Based on the results of RQ1 in Section 5.1, we select the baselines with the best bug detection performance in each category (Time-Machine, ComboDroid, Humanoid, Guardian, BugHunter) to run on a set of real-world popular apps. TraceDroid and the baseline methods are configured with the same running time, with each app running for 60 minutes.

For this experiment, we test 116 apps downloaded from Google Play, and TraceDroid successfully detects 21 previously unknown crash bugs across these apps. Among these, 15 bugs have already been fixed by developers, and 6 bugs have been confirmed and marked for future fixes. In contrast, the best-performing baseline (BugHunter) detects only 6 of these bugs, and all are a strict subset of those identified by TraceDroid. None of the other baselines find any additional bugs beyond those already covered by TraceDroid. We further inspect the triggering paths of these newly discovered crashes and find that they typically require multi-step interactions, with an average length of 8.7 GUI actions. This again highlights the effectiveness of TraceDroid in exposing crash-inducing behaviors that remain hidden from existing automated GUI testing.

Table 5 lists the bugs that are confirmed or fixed by developers, the screenshot of the email is available on our website<sup>1</sup>. The responses from developers further confirm the usefulness of TraceDroid. One developer replied: “Thanks, we were not aware of this bug. It was due to improper handling of null objects in the background service.” These results demonstrate that TraceDroid not only improves bug detection performance under open source apps but also provides practical value in real-world popular apps.

## 6 Discussion

### 6.1 Compatibility with Static Analysis Tools

The first sub-module of TraceDroid (Heuristic-based Suspicious Code Detection) shares some similarities with traditional static analysis tools, Android Lint [1], SpotBugs [10], PMD [9], and Infer [6], which are also designed to scan code and highlight abnormal patterns. To evaluate whether our subsequent modules can enhance the findings of these tools, we replace the first module of

Table 5. Information on the confirmed or fixed bugs. (RQ3)

| ID | App name        | Down  | Category  | Version | Status    | Time | Comb | Huma | Guard | BugH | Bug Type             |
|----|-----------------|-------|-----------|---------|-----------|------|------|------|-------|------|----------------------|
| 1  | Journal         | 5M+   | Product   | 131     | Fixed     |      |      |      |       |      | NullPointerException |
| 2  | Height Increase | 5M+   | Fitness   | 1.1.8   | Fixed     |      |      |      |       |      | RuntimeError         |
| 3  | Offline Music   | 5M+   | Music     | 1.34.1  | Fixed     |      |      |      | *     | *    | NullPointerException |
| 4  | DailyLife       | 5M+   | Lifestyle | 4.3.1   | Fixed     |      |      |      |       |      | ExternalInte         |
| 5  | Easy Notes      | 1M+   | Product   | 1.3.23  | Fixed     |      | *    |      |       | *    | RuntimeError         |
| 6  | ExpenseTracker  | 1M+   | Finance   | 1.01    | Fixed     |      |      |      |       |      | Concurrency          |
| 7  | My Cash         | 1M+   | Enter     | 38      | Confirmed |      |      |      | *     |      | NullPointerException |
| 8  | BetterMe        | 1M+   | Health    | 9.31    | Fixed     |      | *    |      |       | *    | DBOperation          |
| 9  | Daily Yoga      | 1M+   | Sport     | 1.2.1   | Fixed     |      |      |      |       |      | NullPointerException |
| 10 | Daily Routine   | 1M+   | Tool      | 1.1.10  | Confirmed |      |      |      |       |      | RuntimeError         |
| 11 | FreeYourMusic   | 1M+   | Music     | 9.17.0  | Fixed     |      |      | *    |       |      | NullPointerException |
| 12 | WeatherForecast | 1M+   | Weather   | 2.1     | Confirmed |      |      |      |       | *    | RuntimeError         |
| 13 | Add Text        | 1M+   | Photo     | 2.7     | Fixed     | *    |      |      | *     |      | NullPointerException |
| 14 | SideChef        | 1M+   | Food      | 5.31.1  | Fixed     |      |      |      |       |      | ExternalInte         |
| 15 | RISE            | 1M+   | Lifestyle | 1.78.33 | Fixed     |      |      |      |       |      | ExternalInte         |
| 16 | Timetable       | 1M+   | Education | 2.3.1   | Confirmed |      |      |      |       |      | Concurrency          |
| 17 | Daily Planner   | 1M+   | Product   | 63      | Fixed     | *    | *    | *    | *     | *    | RuntimeError         |
| 18 | Headout         | 1M+   | Travel    | 8.19.0  | Confirmed |      |      |      |       |      | ExternalInte         |
| 19 | MyMoney         | 1M+   | Finance   | 5.7     | Fixed     |      |      |      |       |      | DBOperation          |
| 20 | Quabble         | 500K+ | Health    | 2.6.8   | Confirmed |      |      |      |       | *    | Concurrency          |
| 21 | EdrawMind       | 500K+ | Tool      | 7.9.1   | Fixed     |      |      |      |       |      | DBOperation          |

Table 6. Comparison of TraceDroid with mainstream static analysis tools in bug detection

| Metric | Android Lint | SpotBugs | PMD  | Infer | TraceDroid |
|--------|--------------|----------|------|-------|------------|
| #Bugs  | 8            | 10       | 13   | 10    | 54         |
| Recall | 0.11         | 0.14     | 0.19 | 0.14  | 0.77       |

TraceDroid with each of them and then run the rest of our pipeline unchanged. As shown in Table 6, traditional static analysis tools alone detect very few crash bugs, with recalls of 0.11, 0.14, 0.19, and 0.14, respectively. In contrast, TraceDroid achieves a recall of 0.77, which is 58% higher than the best-performing static analysis baseline (PMD). The gap stems from the fact that these tools primarily flag isolated code issues without considering how they can be triggered at runtime. By feeding their outputs into our second and third modules, however, TraceDroid is able to map suspicious segments to GUI components, reconstruct feasible triggering paths, and validate them through dynamic execution. This demonstrates that our framework not only outperforms existing static analysis tools when used alone but can also serve as a general enhancer to amplify their findings in the context of automated GUI testing.

## 6.2 Enhancing LLM-Based GUI Testing with TraceDroid

To integrate TraceDroid with existing LLM-based testers (the “+” variants), we inject the suspicious paths generated by TraceDroid into their prompts. Specifically, instead of replacing their decision-making process, we prepend an additional instruction to the prompt, stating that following the given suspicious paths may help uncover crash bugs, and explicitly list the paths. With this integration, we observe clear improvements across all baselines. The most improvement is observed in Guardian, whose recall rises from 0.39 to 0.51, representing a 31% relative gain. This demonstrates that TraceDroid complements LLM-based methods by injecting targeted suspicious paths into their prompts, thereby reducing exploration randomness and steering them toward higher-value interactions. TraceDroid not only outperforms standalone baselines, but it can also act as a general enhancement layer to strengthen existing LLM-based GUI testing frameworks.

## 6.3 Advantages over Test Case Migration and Bug Replay Techniques

Test case migration (e.g., AppTestMigrator [16], Craftdroid [43], TEMdroid [84]) and bug replay techniques (e.g., RecDroid+[85], ScopeDroid [32], AdbGPT [27]) are designed to reuse existing

Table 7. Performance of integrating TraceDroid into existing LLM-based GUI testing methods

| Metric | TraceDroid  | Tools with TraceDroid |      |      |      |      |      | Tools without TraceDroid |      |      |      |      |      |
|--------|-------------|-----------------------|------|------|------|------|------|--------------------------|------|------|------|------|------|
|        |             | ApA+                  | GPD+ | DRA+ | AUI+ | VID+ | GUA+ | ApA                      | GPD  | DRA  | AUI  | VID  | GUA  |
| #Bugs  | <b>54</b>   | 24                    | 27   | 26   | 29   | 34   | 36   | 21                       | 24   | 20   | 23   | 26   | 27   |
| Recall | <b>0.77</b> | 0.34                  | 0.39 | 0.37 | 0.41 | 0.49 | 0.51 | 0.30                     | 0.34 | 0.29 | 0.33 | 0.37 | 0.39 |

Notes: "ApA" is the AppAgent, "GPD" is the GPTDroid, "DRA" is the DroidAgent, "AUI" is the AUITestAgent, "VID" is the VisionDroid, "GUA" is the Guardian.

execution traces or adapt historical test cases to new app versions. For a fair comparison, we adapt TraceDroid's outputs to the input formats expected by these techniques. Specifically, the suspicious paths generated by our context-aware path generation module are rewritten into migration-style test cases and replay-style step sequences, allowing existing tools to consume them as if they were historical traces. As shown in Table 8, TraceDroid detects 54 crash bugs with a recall of 0.77, substantially outperforming all migration and replay methods, whose recalls remain below 0.31. Even Craftdroid and TEMdroid, detect only 22 bugs (0.31 recall), representing less than half of TraceDroid's detection capability. This demonstrates that directly reasoning from suspicious code-level signals to executable paths provides a more proactive and effective means of discovering crashes than passively adapting past traces.

Table 8. Performance of different test case migration and bug replay methods

| Metric | RecDroid+ | ScopeDroid | AdbGPT | AppTestMigrator | Craftdroid | TEMdroid | TraceDroid  |
|--------|-----------|------------|--------|-----------------|------------|----------|-------------|
| #Bugs  | 18        | 19         | 17     | 20              | 22         | 22       | <b>54</b>   |
| Recall | 0.26      | 0.27       | 0.24   | 0.29            | 0.31       | 0.31     | <b>0.77</b> |

#### 6.4 Threats of Validity

The first threat concerns the representativeness of the studied apps and bugs. In RQ1 and RQ2, we used 42 apps with 70 confirmed crash bugs, and in RQ3 we selected additional popular Google Play apps. Although these datasets cover diverse categories, some domains or crash patterns may still be underrepresented. The second threat is the potential influence of LLM pretraining. Since LLMs are trained on large corpora, some apps or code snippets similar to ours may have appeared in training. To mitigate this, we excluded apps from our pilot study and chose recently updated Google Play apps. The third threat relates to LLM hallucinations when generating paths or actions. LLMs may produce non-existent widgets or infeasible transitions. TraceDroid employs lightweight validators after each generation step: ATG consistency, widget existence, and action schema, discarding or repairing invalid outputs. These safeguards substantially reduce the impact of hallucinations, but cannot completely eliminate the risk of subtle semantic errors. The fourth threat concerns the completeness of the statically constructed Activity Transition Graph (ATG) and its interaction with LLM-based path generation. Some dynamic transitions, such as those introduced via reflection-based routing or data-dependent implicit intents, may not be fully captured by static analysis. In our evaluation, 92% of inter-activity transitions observed during dynamic execution are already present in the ATG, while the remaining 8% fall into these hard-to-analyze cases. For LLM-generated paths, 14% require at least one deterministic repair (e.g., replacing an unreachable hop with a shortest valid path in the ATG), and 6% are discarded due to invalid widget references. These cases are effectively handled by validation and repair mechanisms before execution, but extreme cases of highly dynamic navigation may still pose challenges.

## 7 Related Work

**Automated GUI Testing.** To ensure the quality of mobile apps, many researchers study the automatic generation of large-scale test scripts to test apps [76]. Since Android apps are event-based [12, 33, 52, 75], the most common automated testing approaches are model-based automated GUI testing approaches [25, 28, 51, 53, 63, 71, 78, 80, 83], which design corresponding models through the analysis of the apps. Due to the lack of consideration for the semantic information of the app's GUI pages, the coverage of model-based approaches is still low. Researchers further proposed human-like testing strategies and designed LLM-based automated GUI testing approaches. GPTDroid [45] used GPT-3.5 to generate the testing script. AppAgent [81] enabled the agent to operate apps through a simplified action space. DroidAgent [82] generated high-level, realistic tasks for GUI testing based on app-specific functionalities. AUITestAgent [31] is used for step-oriented testing. AXNav [67] generated annotated videos to visually and interactively review accessibility test results. UXAgent [49] achieved LLM-driven automated usability testing by simulating thousands of user interactions. TestAgent [40] generated and executed test cases through multimodal perception. Although these methods improve coverage and interaction diversity, they remain limited in efficiently detecting crash-inducing behaviors.

**Static Program Analysis.** Static analysis [24, 35–38, 50, 54, 56, 57, 59–61, 74, 77, 79] is another widely adopted technique to improve software reliability. Researchers have developed tools targeting different categories of bugs. For null pointer dereference [68], Infer [6] and NullAway [8] analyzed type annotations and data-flow conditions to detect potential unsafe accesses. For concurrency errors [15, 22, 30], RacerD [17] identified race conditions and missing synchronization in shared-memory programs. For resource management [18, 21, 34, 70] and API misuse [11, 11, 39, 50], FindBugs [5] and SpotBugs [10] provide rule-based checks to detect memory leaks, lifecycle misuses, and incorrect Android API calls.

## 8 Conclusion

Detecting crash bugs in mobile apps remains a challenging problem, as many crashes require precise GUI interaction sequences that are difficult to reach with existing automated testing methods. In this paper, we propose TraceDroid, a novel automated GUI testing approach that leverages suspicious code-level signals to guide exploration toward crash-prone interactions. TraceDroid first detects suspicious code segments and associates them with GUI widgets to collect code-level interaction signals. It then constructs the ATG, generates rough paths, and employs LLM-based executable path completion to anchor suspicious interactions to feasible execution routes. Finally, TraceDroid executes these suspicious paths through global path planning, local path generation, and execution-aware monitoring, enabling efficient and reliable crash detection. The experimental results demonstrate the effectiveness of TraceDroid. These results confirm that TraceDroid can effectively transform suspicious static traces into actionable execution paths, improving both the efficiency and practicality of automated GUI testing.

In the future, we plan to extend TraceDroid to cover a wider range of abnormal behaviors beyond crashes, and to investigate tighter integration with emerging LLM-based GUI testing methods for further enhancing its adaptability and bug detection capability.

## 9 Data Availability

We release the source code, dataset, and experimental results on our website <https://github.com/TheMystery123/TraceDroid>.

## Acknowledgments

This work was supported by the National Natural Science Foundation of China Grant No. 62402483, 62232016, 62072442, 62402484, Youth Innovation Promotion Association CAS, Basic Research Program of ISCAS Grant No. ISCAS-JCZD-202304, Major Program of ISCAS Grant No. ISCAS-ZD-202302 and ISCAS-ZD-202401, Innovation Team 2024 ISCAS (No. 2024-66).

## References

- [1] 2020. <http://tools.android.com/tips/lint>.
- [2] 2020. Android Debug Bridge (adb). <https://developer.android.com/studio/command-line/adb>.
- [3] 2022. Android. <https://developer.android.google/topic/>.
- [4] 2023. pyvbox. <https://pypi.org/project/pyvbox/>.
- [5] 2025. Findbugs. <https://findbugs.sourceforge.net/>.
- [6] 2025. Infer. <https://fbinfer.com/>.
- [7] 2025. Libretube. <https://libretube.dev/>.
- [8] 2025. NullAway. <https://www.baeldung.com/java-nullaway>.
- [9] 2025. PMD. <https://docs.pmd-code.org/latest/>.
- [10] 2025. SpotBugs. <https://spotbugs.readthedocs.io/en/latest/introduction.html>.
- [11] Sven Amann. 2018. *A systematic approach to benchmark and improve automated static detection of Java-API misuses*. Ph.D. Dissertation. Universitäts- und Landesbibliothek Darmstadt.
- [12] Saswat Anand, Mayur Naik, Mary Jean Harrold, and Hongseok Yang. 2012. Automated concolic testing of smartphone apps. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*. 1–11.
- [13] Yauhen Leanidavich Arnatovich, Lipo Wang, Ngoc Minh Ngo, and Charlie Soh. 2018. Mobolic: An automated approach to exercising mobile application GUIs using symbiosis of online testing technique and customized input generation. *Software: Practice and Experience* 48, 5 (2018), 1107–1142.
- [14] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *Acm Sigplan Notices* 49, 6 (2014), 259–269.
- [15] Manuel Barros, Maria Ramos, Alexandre Gomes, Alcino Cunha, José Pereira, and Paulo Sérgio Almeida. 2023. An experimental evaluation of tools for grading concurrent programming exercises. In *International Conference on Formal Techniques for Distributed Objects, Components, and Systems*. Springer, 3–20.
- [16] Farnaz Behrang and Alessandro Orso. 2019. Test migration between mobile apps with similar functionality. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 54–65.
- [17] Sam Blackshear, Nikos Gorogiannis, Peter W. O’Hearn, and Ilya Sergey. 2018. RacerD: compositional static race detection. *Proc. ACM Program. Lang.* (2018).
- [18] Marco Brocanelli. 2018. *Improving the performance of smartphone apps with soft hang bug detection and dynamic resource management*. Ph.D. Dissertation. The Ohio State University.
- [19] Tianqin Cai, Zhao Zhang, and Ping Yang. 2020. Fastbot: A Multi-Agent Model-Based Test Generation System Beijing Bytedance Network Technology Co., Ltd.. In *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*. 93–96.
- [20] Mengzhuo Chen, Zhe Liu, Chunyang Chen, Junjie Wang, Boyu Wu, Jun Hu, and Qing Wang. 2025. Standing on the Shoulders of Giants: Bug-Aware Automated GUI Testing via Retrieval Augmentation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 825–846.
- [21] Mohan Cui, Chengjun Chen, Hui Xu, and Yangfan Zhou. 2023. SafeDrop: Detecting memory deallocation bugs of rust programs via static data-flow analysis. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–21.
- [22] Zhi Da Luo, Linda Hillis, Raja Das, and Yao Qi. 2010. Effective static analysis to find concurrency bugs in Java. In *2010 10th IEEE Working Conference on Source Code Analysis and Manipulation*. IEEE, 135–144.
- [23] Android Developers. 2012. Ui/application exerciser monkey.
- [24] Malinda Dilhara, Haipeng Cai, and John Jenkins. 2018. Automated detection and repair of incompatible uses of runtime permissions in android apps. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*. 67–71.
- [25] Zhen Dong, Marcel Böhme, Lucia Cojocar, and Abhik Roychoudhury. 2020. Time-travel testing of android apps. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. IEEE, 481–492.
- [26] Lingling Fan, Ting Su, Sen Chen, Guozhu Meng, Yang Liu, Lihua Xu, Geguang Pu, and Zhendong Su. 2018. Large-scale analysis of framework-specific exceptions in android apps. In *2018 IEEE/ACM 40th International Conference on Software*

- Engineering (ICSE)*. IEEE, 408–419.
- [27] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [28] Tianxiao Gu, Chengnian Sun, Xiaoxing Ma, Chun Cao, Chang Xu, Yuan Yao, Qirun Zhang, Jian Lu, and Zhendong Su. 2019. Practical GUI testing of Android applications via model abstraction and refinement. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 269–280.
- [29] Yuyu He, Lei Zhang, Zhemin Yang, Yinzhi Cao, Keke Lian, Shuai Li, Wei Yang, Zhibo Zhang, Min Yang, Yuan Zhang, et al. 2020. TextExerciser: feedback-driven text input exercising for android applications. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1071–1087.
- [30] Shin Hong, Yongbae Park, and Moonzoo Kim. 2014. Detecting concurrency errors in client-side java script web applications. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, 61–70.
- [31] Yongxiang Hu, Xuan Wang, Yingchuan Wang, Yu Zhang, Shiyu Guo, Chaoyi Chen, Xin Wang, and Yangfan Zhou. 2024. Auitestagent: Automatic requirements oriented gui function testing. *arXiv preprint arXiv:2407.09018* (2024).
- [32] Yuchao Huang, Junjie Wang, Zhe Liu, Song Wang, Chunyang Chen, Mingyang Li, and Qing Wang. 2023. Context-aware bug reproduction for mobile apps. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2336–2348.
- [33] Reyhaneh Jabbarvand, Jun-Wei Lin, and Sam Malek. 2019. Search-based energy testing of android. In *ICSE*. IEEE, 1119–1130.
- [34] Hao Jiang, Hongli Yang, Shengchao Qin, Zhendong Su, Jian Zhang, and Jun Yan. 2017. Detecting energy bugs in android apps using static analysis. In *International Conference on Formal Engineering Methods*. Springer, 192–208.
- [35] Kadiray Karakaya and Eric Bodden. 2021. Sootfx: A static code feature extraction tool for java and android. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 181–186.
- [36] Chaoran Li, Xiao Chen, Ruoxi Sun, Minhui Xue, Sheng Wen, Muhammad Ejaz Ahmed, Seyit Camtepe, and Yang Xiang. 2022. Cross-language android permission specification. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 772–783.
- [37] Li Li, Alexandre Bartel, Tegawendé F Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Outeau, and Patrick McDaniel. 2015. Iccta: Detecting inter-component privacy leaks in android apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 280–291.
- [38] Li Li, Tegawendé F Bissyandé, Damien Outeau, and Jacques Klein. 2016. Droidra: Taming reflection to support whole-program analysis of android apps. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. 318–329.
- [39] Xia Li, Jiajun Jiang, Samuel Benton, Yingfei Xiong, and Lingming Zhang. 2021. A Large-scale Study on API Misuses in the Wild. In *2021 14th IEEE conference on software testing, verification and validation (ICST)*. IEEE, 241–252.
- [40] Youwei Li, Yangyang Li, and Yangzhao Yang. 2024. Test-Agent: A Multimodal App Automation Testing Framework Based on the Large Language Model. In *2024 IEEE 4th International Conference on Digital Twins and Parallel Intelligence (DTPI)*. IEEE, 609–614.
- [41] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2017. Droidbot: a lightweight ui-guided test input generator for android. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 23–26.
- [42] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2019. Humanoid: a deep learning-based approach to automated black-box Android app testing. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1070–1073.
- [43] Jun-Wei Lin, Reyhaneh Jabbarvand, and Sam Malek. 2019. Test transfer across mobile apps through semantic mapping. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 42–53.
- [44] Peng Liu, Xiangyu Zhang, Marco Pistoia, Yunhui Zheng, Manoel Marques, and Lingfei Zeng. 2017. Automatic text input generation for mobile testing. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 643–653.
- [45] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Chatting with gpt-3 for zero-shot human-like mobile automated gui testing. *arXiv preprint arXiv:2305.09434* (2023).
- [46] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2024. Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [47] Zhe Liu, Chunyang Chen, Junjie Wang, Yuhui Su, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Ex pede Herculem: Augmenting Activity Transition Graph for Apps via Graph Convolution Network. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1983–1995.
- [48] Zhe Liu, Cheng Li, Chunyang Chen, Junjie Wang, Boyu Wu, Yawen Wang, Jun Hu, and Qing Wang. 2024. Vision-driven automated mobile gui testing via multimodal large language model. *arXiv preprint arXiv:2407.03037* (2024).

- [49] Yuxuan Lu, Bingsheng Yao, Hansu Gu, Jing Huang, Jessie Wang, Yang Li, Jiri Gesi, Qi He, Toby Jia-Jun Li, and Dakuo Wang. 2025. Uxagent: An llm agent-based usability testing framework for web design. *arXiv preprint arXiv:2502.12561* (2025).
- [50] Tarek Mahmud, Meiru Che, and Guowei Yang. 2022. Acid: an api compatibility issue detector for android apps. In *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings*. 1–5.
- [51] Ke Mao, Mark Harman, and Yue Jia. 2016. Sapienz: Multi-objective automated testing for Android applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. 94–105.
- [52] Reza Matinnejad, Shiva Nejati, and Lionel C Briand. 2017. Automated testing of hybrid Simulink/Stateflow controllers: industrial case studies. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 938–943.
- [53] Nariman Mirzaei, Joshua Garcia, Hamid Bagheri, Alireza Sadeghi, and Sam Malek. 2016. Reducing combinatorics in GUI testing of android applications. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. IEEE, 559–570.
- [54] Lucky Onwuzurike, Enrico Mariconti, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2019. Mamadroid: Detecting android malware by building markov chains of behavioral models (extended version). *ACM Transactions on Privacy and Security (TOPS)* 22, 2 (2019), 1–34.
- [55] Minxue Pan, An Huang, Guoxin Wang, Tian Zhang, and Xuandong Li. 2020. Reinforcement learning based curiosity-driven testing of Android applications. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 153–164.
- [56] Felix Pauck and Heike Wehrheim. 2021. Jicer: Simplifying cooperative android app analysis tasks. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 187–197.
- [57] Lina Qiu, Yingying Wang, and Julia Rubin. 2018. Analyzing the analyzers: Flowdroid/iccta, amandroid, and droidsafe. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 176–186.
- [58] Dezhi Ran, Hao Wang, Zihe Song, Mengzhou Wu, Yuan Cao, Ying Zhang, Wei Yang, and Tao Xie. 2024. Guardian: A Runtime Framework for LLM-Based UI Exploration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 958–970.
- [59] Jordan Samhi, Alexandre Bartel, Tegawendé F Bissyandé, and Jacques Klein. 2021. Raicc: Revealing atypical inter-component communication in android apps. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1398–1409.
- [60] Jordan Samhi, René Just, Tegawendé F Bissyandé, Michael D Ernst, and Jacques Klein. 2024. Call graph soundness in android static analysis. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 945–957.
- [61] Jordan Samhi, Li Li, Tegawendé F Bissyandé, and Jacques Klein. 2022. Difuzer: Uncovering suspicious hidden sensitive operations in android apps. In *Proceedings of the 44th International Conference on Software Engineering*. 723–735.
- [62] Donna Spencer. 2009. *Card sorting: Designing usable categories*. Rosenfeld Media.
- [63] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, stochastic model-based GUI testing of Android apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 245–256.
- [64] Ting Su, Jue Wang, and Zhendong Su. 2021. Benchmarking automated GUI testing for Android against real-world bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 119–130.
- [65] Yanqi Su, Zheming Han, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2022. Constructing a system knowledge graph of user tasks and failures from bug reports to support soap opera testing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [66] Yanqi Su, Dianshu Liao, Zhenchang Xing, Qing Huang, Mulong Xie, Qinghua Lu, and Xiwei Xu. 2024. Enhancing exploratory testing by large language model and knowledge graph. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [67] Maryam Taeb, Amanda Swearngin, Eldon Schoop, Ruijia Cheng, Yue Jiang, and Jeffrey Nichols. 2024. Axnav: Replaying accessibility tests from natural language. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [68] David A Tomassi and Cindy Rubio-González. 2021. On the real-world effectiveness of static bug detectors at finding null pointer exceptions. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 292–303.
- [69] UIAutomator. 2021. Python wrapper of Android uiautomator test tool. <https://github.com/xiaocong/uiautomator>.
- [70] Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2025. Boosting static resource leak detection via llm-based resource-oriented intention inference. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 668–668.

- [71] Jue Wang, Yanyan Jiang, Chang Xu, Chun Cao, Xiaoxing Ma, and Jian Lu. 2020. Combodroid: generating high-quality test inputs for android apps via use case combinations. In *ICSE*. 469–480.
- [72] Jun Wang, Yanhui Li, Zhifei Chen, Lin Chen, Xiaofang Zhang, and Yuming Zhou. 2024. Knowledge Graph Driven Inference Testing for Question Answering Software. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [73] Wenyu Wang, Wei Yang, Tianyin Xu, and Tao Xie. 2021. Vet: identifying and avoiding UI exploration tar pits. In *FSE*. 83–94.
- [74] Daoyuan Wu, Debin Gao, Robert H Deng, and Chang Rocky KC. 2021. When program analysis meets bytecode search: Targeted and efficient inter-procedural analysis of modern Android apps in BackDroid. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 543–554.
- [75] Tianyong Wu, Xi Deng, Jun Yan, and Jian Zhang. 2019. Analyses for specific defects in android applications: A survey. *Frontiers of Computer Science* (2019), 1–18.
- [76] Qing Xie and Atif M Memon. 2007. Designing and comparing automated test oracles for GUI-based software applications. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 16, 1 (2007), 4–es.
- [77] Jiwei Yan, Hao Liu, Linjie Pan, Jun Yan, Jian Zhang, and Bin Liang. 2020. Multiple-entry testing of android applications by constructing activity launching contexts. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. IEEE, 457–468.
- [78] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. 2018. Static window transition graphs for Android. *Automated Software Engineering* 25, 4 (2018), 833–873.
- [79] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. 2015. Static control-flow analysis of user-driven callbacks in Android applications. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 89–99.
- [80] Wei Yang, Mukul R Prasad, and Tao Xie. 2013. A grey-box approach for automated GUI-model generation of mobile applications. In *International Conference on Fundamental Approaches to Software Engineering*. Springer, 250–265.
- [81] Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. 2023. Appagent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771* (2023).
- [82] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous large language model agents enabling intent-driven mobile gui testing. *arXiv preprint arXiv:2311.08649* (2023).
- [83] Xia Zeng, Dengfeng Li, Wujie Zheng, Fan Xia, Yuetang Deng, Wing Lam, Wei Yang, and Tao Xie. 2016. Automated test input generation for android: Are we really there yet in an industrial case?. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 987–992.
- [84] Yakun Zhang, Wenjie Zhang, Dezhi Ran, Qihao Zhu, Chengfeng Dou, Dan Hao, Tao Xie, and Lu Zhang. 2024. Learning-based Widget Matching for Migrating GUI Test Cases. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [85] Yu Zhao, Ting Su, Yang Liu, Wei Zheng, Xiaoxue Wu, Ramakanth Kavuluru, William GJ Halfond, and Tingting Yu. 2022. ReCDroid+: Automated End-to-End Crash Reproduction from Bug Reports for Android Apps. *TOSEM* (2022).

Received 2025-09-12; accepted 2026-03-24